TuHospedaje

INFORME DE SPRINT

Bitácora de Ejecución y Cierre — Sprint 1

TuHospedaje — Estructura Base y Catálogo

HISTORIAS

US #1 a #11

TEMA

Base del sistema: catálogo, galería y panel de administración

INICIO DEL REPOSITORIO

2026-05-18 (commit 74cb8a2)

Equipo de Desarrollo · Mayo 2026

Foco del Incremento: Estructura Base del Sistema y Catálogo de Alojamientos **Stack Tecnológico:** Java 17 / Spring Boot 3.5 / MariaDB / React 19 / Vite

> ***Nota:*** *Este documento describe el estado del proyecto al cierre del Sprint 1. Las referencias a endpoints sin autenticación reflejan el estado original del sistema, antes de la implementación de seguridad en Sprint 2.*

1. Resumen del Incremento (Scope)

El Objetivo del Sprint 1 consistió en establecer las bases de la arquitectura de software (frontend y backend), desplegar el modelo relacional inicial y proveer un catálogo funcional de alojamientos. Al cierre del sprint, la solución permite a los usuarios visualizar opciones en la interfaz pública y faculta a los administradores a dar de alta, listar con paginación y eliminar hospedajes a través de un panel de control dedicado.

2. Arquitectura del Sistema e Integración

2.1. Backend (Spring Boot)

Se implementó una arquitectura en capas con un flujo unidireccional y desacoplado: `Controller → Service (Interface + Impl) → Repository → Entity / DTO`. El detalle vigente de esa arquitectura está en backend-architecture.md; este reporte no la redefine.

- **Desacoplamiento de Datos:** Intercambio de información gestionado mediante una entidad unificada `LodgingDTO` que encapsula la lógica de conversión mediante los métodos estáticos `toEntity()` y `fromEntity()`.
- **Exposición de Imágenes:** El `LodgingDTO` expone `List<String> imageUrls` mapeado desde la relación `@OneToMany` con `LodgingImage`, permitiendo al frontend consumir las URLs sin acoplamiento a la entidad.
- **Inyección de Dependencias:** Inyección estricta basada en constructores, prescindiendo de `@Autowired` directo en atributos para asegurar la testabilidad unitaria. Automatización de boilerplate mediante Lombok.
- **Política de CORS:** Configuración explícita en la capa de red para permitir peticiones provenientes del puerto de desarrollo de frontend (`http://localhost:5173`), autorizando de forma nativa los métodos de pre-vuelo (`OPTIONS`).
- **Seguridad Temporal:** El ciclo de filtros se configuró para mantener todos los endpoints bajo la ruta `/api/` como de acceso público exclusivo durante este corte.
- **Manejo Global de Errores:** Se implementó `GlobalExceptionHandler` con `@RestControllerAdvice` que captura `ResourceNotFoundException` → 404 e `IllegalArgumentException` → 400, centralizando el manejo de errores HTTP.

2.2. Frontend (React + Vite)

Estructura modularizada basada en componentes reutilizables y vistas de ruteo:

```
src/
├── components/   (Header, Footer, ProductCard)
├── pages/        (Home, ProductDetail, Admin)
└── services/     (api.js — módulo fetch centralizado)
```

- **Navegación:** Declarativa mediante React Router (Single Page Application).
- **Gestión de Estilos:** CSS Puro con Variables Dinámicas (Custom Properties) preparadas para el intercambio de temas Light/Dark.
- **Control de Entorno:** Aislamiento de variables de infraestructura (`VITE_API_URL`) mediante archivos `.env` .

3. Trazabilidad de Historias de Usuario (User Stories)

ID	HISTORIA DE USUARIO	COMPONENTE / VISTA UI	ENDPOINT ASOCIADO	CRITERIO DE ACEPTACIÓN / ESTADO
US #1	Encabezado (Header) funcional con logo, lema corporativo y botones de navegación.	`Header.jsx`	N/A (Navegación local)	Elemento fijo (fixed) superior en el 100% de las vistas. Redirección al Home mediante clic.
US #2	Estructura principal del Home: buscador, categorías y grilla.	`Home.jsx`	`GET /api/lodgings/random`	Carga de placeholders estáticos para el buscador/categorías y consumo dinámico para las recomendaciones.
US #3	Registro de hospedaje con validaciones desde vista de administración.	`Admin.jsx` (Modal)	`POST /api/lodgings`	Formulario emergente con validación de tipos; persistencia inmediata en MariaDB.
US #4	Despliegue de grilla aleatoria de 10 recomendaciones en Home.	`ProductCard.jsx`	`GET /api/lodgings/random`	Renderizado responsivo en formato de grilla CSS (4 columnas en desktop).
US #5	Visualización detallada de la ficha técnica del alojamiento.	`ProductDetail.jsx`	`GET /api/lodgings/{id}`	Recuperación por ID. Enrutamiento dinámico `/lodging/:id` .
US #6	Galería multimedia interactiva en el detalle del producto.	`ProductDetail.jsx`	Contenido de `Lodging.images`	Layout simétrico (50/50) con grilla adaptativa y botón disparador "Ver más".

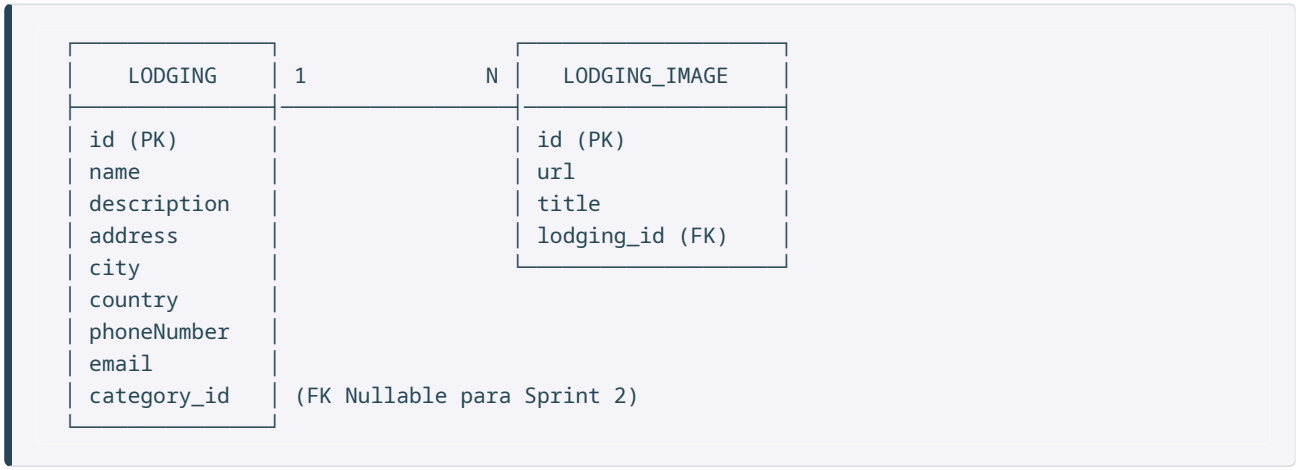
US #7	Pie de página (Footer) con derechos de autor e identidad de marca.	`Footer.jsx`	N/A (UI Estática)	Consistente y posicionado al final del scroll en todo el sitio.
US #8	Mecanismo de paginación de catálogo en el panel de control.	`Admin.jsx`	`GET /api/lodgings?page=&size=`	Controles de navegación "Anterior", "Siguiente" e "Inicio". Tamaño fijado en 10 registros.
US #9	Menú de navegación y layout del Panel de Administración.	`Admin.jsx`	Ruteo `/admin`	Acceso directo a funciones CRUD de catálogo.
US #10	Tabla de visualización de inventario para el Administrador.	`Admin.jsx` (Tabla)	`GET /api/lodgings`	Despliegue estructurado: ID, Nombre, Ubicación y columna de Acciones Directas.
US #11	Baja física de un alojamiento del catálogo.	`Admin.jsx`	`DELETE /api/lodgings/{id}`	Disparador preventivo mediante `window.confirm`. Borrado y refresco asíncrono de la UI.

4. Catálogo de Endpoints de la API REST

MÉTODO	ENDPOINT	PARÁMETROS DE ENTRADA	DESCRIPCIÓN / COMPORTAMIENTO TÉCNICO
POST	`/api/lodgings`	Body: `LodgingDTO` (JSON)	Registra un nuevo alojamiento en el sistema y persiste sus URL multimedia.
GET	`/api/lodgings`	Query (Opcional): `page`, `size`	Recupera el catálogo completo de forma paginada para la vista del administrador.
GET	`/api/lodgings/random`	Ninguno	Retorna una colección desordenada de un máximo de 10 alojamientos para el Home.
GET	`/api/lodgings/search`	Query (Obligatorio): `query`	Endpoint habilitado en backend; busca coincidencias parciales por nombre.
GET	`/api/lodgings/{id}`	Path Variable: `id` (Long)	Retorna la ficha técnica y la colección de imágenes del alojamiento consultado.
PUT	`/api/lodgings/{id}`	Path Variable: `id` (Long), Body: `LodgingDTO` (JSON)	Actualiza los campos de un alojamiento existente identificado por su ID.
DELETE	`/api/lodgings/{id}`	Path Variable: `id` (Long)	Ejecuta el borrado físico del registro en base de datos en cascada (`CascadeType.ALL`).

5. Modelo de Datos y Cardinalidad

El esquema relacional implementado en MariaDB se compone de dos entidades nucleares vinculadas de forma estricta:



Relación `Lodging (1)` → `(N) LodgingImage` : Configurada mediante `@OneToMany(mappedBy = "lodging", cascade = CascadeType.ALL, orphanRemoval = true)` en la entidad padre. Esto garantiza que la eliminación de un alojamiento (US #11) destruya de forma limpia y automática todos los registros de imágenes asociados en la base de datos sin dejar registros huérfanos.

6. Decisiones Técnicas Clave

- **Semántica del Negocio (Lodging):** Se adoptó el término técnico e identitario de la industria hotelera en sustitución del genérico `Product` en la totalidad del código fuente, lo cual optimiza la legibilidad del dominio de software.
- **Estrategia Migratoria Preventiva:** La propiedad `Lodging.category` fue declarada como clave foránea nullable desde este sprint. Esto previene la necesidad de ejecutar scripts de migración complejos (DIF / DDL) al momento de introducir la lógica de categorías en el Sprint 2.
- **Arquitectura de Interfaz Restringida:** Siguiendo el criterio de diseño de la aplicación, el panel de administración se diseñó exclusivamente para pantallas de escritorio. Se implementó una lógica de detección táctil (`ontouchstart`) en `Admin.jsx` : si un dispositivo móvil intenta acceder a la ruta `/admin` , el sistema interrumpe la carga y despliega un mensaje explícito indicando que la administración requiere una pantalla optimizada de escritorio.

7. Limitaciones Conocidas y Gestión de Deuda Técnica Controlada

De acuerdo con el enfoque de desarrollo ágil e incremental, se documentan los siguientes placeholders funcionales que serán resueltos en los ciclos subsecuentes:

- **Tratamiento Multimedia Simplificado:** Las imágenes de los alojamientos se gestionan mediante URLs externas de picsum.photos (59 imágenes sembradas para los 14 alojamientos, 3-4 cada uno). Las URLs son deterministas y funcionan en cualquier entorno. La carga real de archivos a un almacenamiento en la nube queda fuera de este entregable.
- **Interactividad de Búsqueda y Filtrado Suspendida:** El bloque del buscador y los botones de categorías en el Home operan únicamente como componentes visuales estáticos (placeholders de

UI). Su interconexión con los motores de indexación asíncrona y lógica de bases de datos se activará en los Sprints 2 y 3.

- **Ausencia de Autenticación:** Los flujos CRUD de administración no requieren credenciales ni tokens de sesión. La protección mediante Spring Security y JSON Web Tokens (JWT) será la prioridad del Sprint 2.
- **Validaciones de Backend Básicas:** Las restricciones de integridad se manejan a nivel de persistencia de datos (campos requeridos) y tipos de datos en los DTOs, aplazando el uso de validaciones avanzadas (como `@Valid` / JSR-380) para los próximos incrementos.